Lab 5: Branch Predictor Lab

In this lab, you will use the SUSTemu RISC-V simulator to explore how branch prediction affects out-of-order processor performance. By the end of this lab, you will be able to:

1. Identify BTB aliasing and explain how function alignment causes it
2. Measure the cold-start penalty of the gshare predictor and explain the effect of GHR width
3. Design a custom branch predictor that outperforms the tournament baseline

Default predictor configuration (`include/cpu/bpred.h`):

- BTB: **512** entries · index = `PC[10:2]` · tag = `PC >> 11`
- Local predictor: LHT **1024** entries × **10-bit** history → LPHT **1024** 2-bit saturating counters
- Global predictor (gshare): GHR **12 bits** XOR PC → GPHT **4096** 2-bit saturating counters
- Meta selector: **4096** 2-bit saturating counters; ≥ 2 → prefer global, < 2 → prefer local

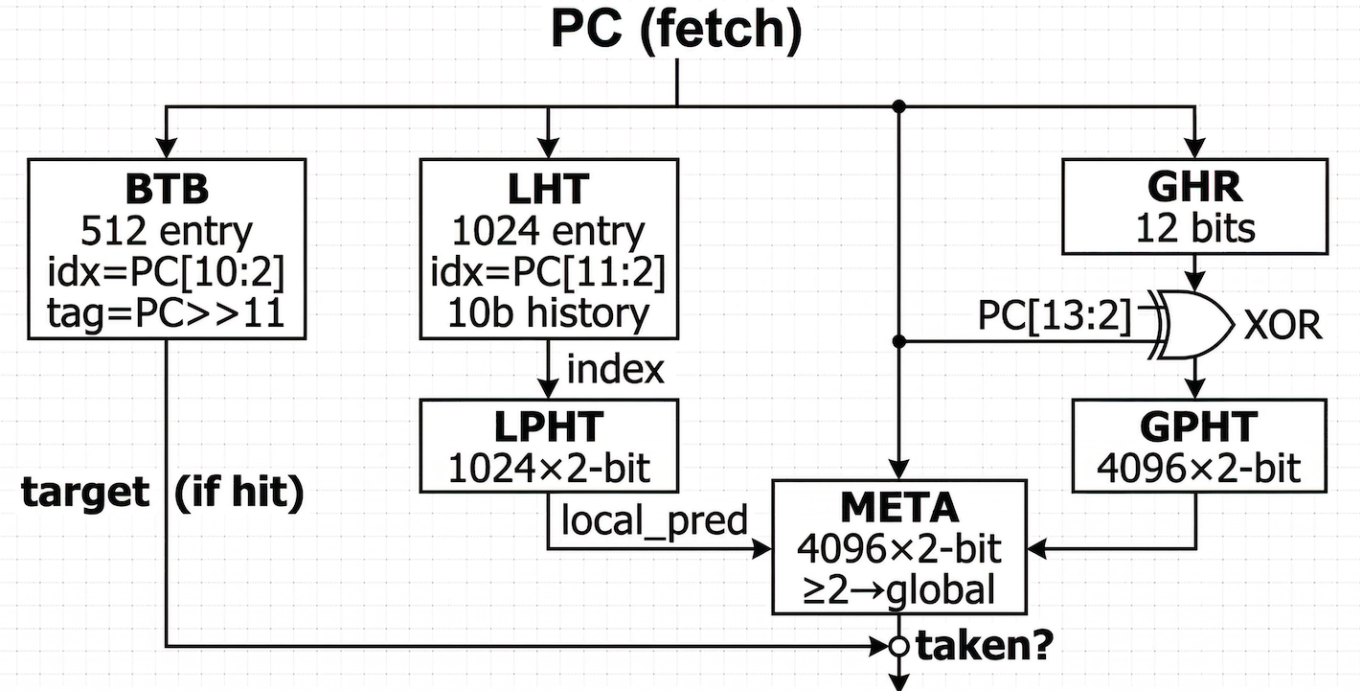
Each benchmark directory provides two (or three) make targets:

```
make run-nobpred   # OOO pipeline, always predict not-taken
make run-bpred     # OOO pipeline, tournament predictor enabled
make run-ooo       # OOO pipeline, tournament predictor enabled (Q3)
make disasm        # Generate annotated disassembly bench.dis
```

> **Note:** after modifying any file under `include/`, always run `make clean && make` from the repository root.

Step 1 — Branch Predictor Effect

Background



`predictor_lab/Q1/` contains a kernel `branch_regular` where taken/not-taken outcomes strictly alternate (50/50). Because a 2-bit saturating counter oscillates between two states on every branch, mispredictions occur frequently regardless of whether the predictor is enabled. Comparing runs with and without the predictor gives a direct view of how branch prediction affects IPC and misprediction count.

```
cd predictor_lab/Q1
make run-nobpred
make run-bpred
```

Question 1: Submit screenshots of both runs showing IPC and Branch Prediction Statistics.

Step 2 — BTB Aliasing

Background

`predictor_lab/Q2/` contains two functions:

- `loop_A` — sums `aa[N]`; `__attribute__((noinline))`
- `loop_B` — sums `bb[N]`; `__attribute__((noinline, aligned(2048)))`

The BTB has **512** entries indexed by `PC[10:2]` with tag `PC >> 11`. `main` calls both functions alternately for `PASSES = 100` rounds.

```
cd predictor_lab/Q2
make run-bpred
```

Question 2: Generate the disassembly and locate the back-edge branch address of each loop:

```
make disasm   # writes bench.dis
```

Fill in:

- `loop_A` back-edge PC = `0x_____`, BTB index = `____`
- `loop_B` back-edge PC = `0x_____`, BTB index = `____`

Do the two indices collide? What is the performance cost of a BTB tag mismatch?

Question 3: Remove `aligned(2048)` from `loop_B`, recompile, and run `make run-bpred`. Compare the **BTB cold misses** before and after the change, and explain why aliasing disappears.

Step 3 — Cold Start

Background

`predictor_lab/Q3/` measures the cold-start penalty of the gshare predictor. All 2-bit saturating counters are initialized to 0 (strongly not-taken). `probe_branches()` contains 32 always-taken branches measured in two states: **cold** (PHT slots all at 0) and **warm** (after 8 warm-up rounds, PHT slots saturated at 3). Between the two measurements, `reset_ghr()` clears the GHR so that gshare indexes the same PHT slots in both passes.

```
cd predictor_lab/Q3
make run-ooo
```

Question 4: Record the `cold` and `warm` cycle counts from the output. Explain why the cold pass takes significantly more cycles than the warm pass.

Step 4 — Design Your Own Branch Predictor

Background

`predictor_lab/Q4/bench.c` contains an inner loop whose conditional branch follows a strictly alternating T/NT/T/NT pattern. The tournament predictor achieves only ~82% accuracy on this workload. The root cause lies in the OOO pipeline: multiple in-flight instances of the same branch are predicted before the previous instance resolves in the EX stage. Because the history register is only updated at EX, consecutive predictions read the same stale history and always predict the same direction — yielding a 50% miss rate on the conditional branch.

Your task is to implement `bpred2_predict` / `bpred2_update` in `src/cpu/bpred2.c` so that accuracy **exceeds** the tournament predictor (> 82.44%). All data structures (BTB, LHT, LPHT, GHR, GPHT, META) are declared in `include/cpu/bpred2.h`. **Do not modify bench.c.**

```
cd predictor_lab/Q4
make run-bpred    # tournament baseline
make run-bpred2   # your design
```

> 💡 **Hint:** You are encouraged to use AI assistants (e.g. Claude, ChatGPT) to help design your predictor. Try describing the problem — OOO pipeline, EX-stage history update, multiple in-flight instances of the same branch — and ask the AI to identify the root cause and suggest a predictor structure. In your write-up, explain the key insight the AI provided and how you validated and debugged the implementation.

Question 5: Submit your `bpred2.c` implementation along with screenshots of `make run-bpred` and `make run-bpred2`. Compare misprediction count and IPC between the two predictors. Explain the core idea behind your design and why it addresses the weakness of the tournament predictor.